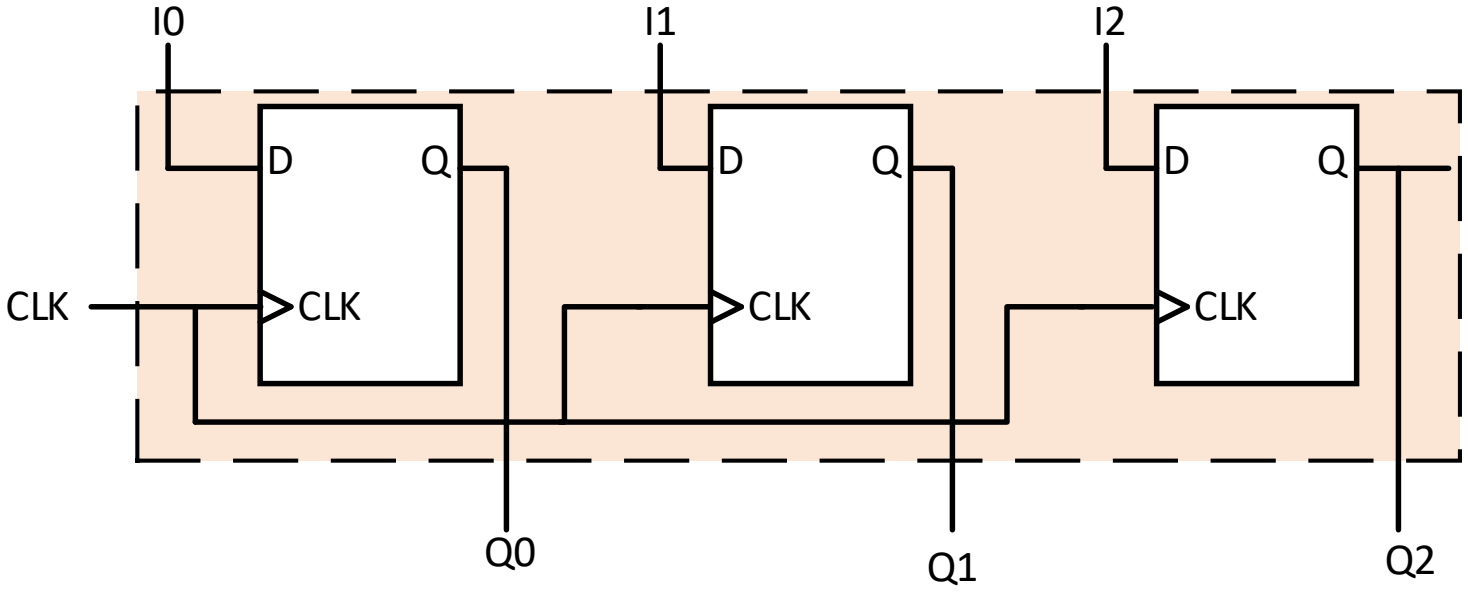
Circuitos Sequenciais Típicos

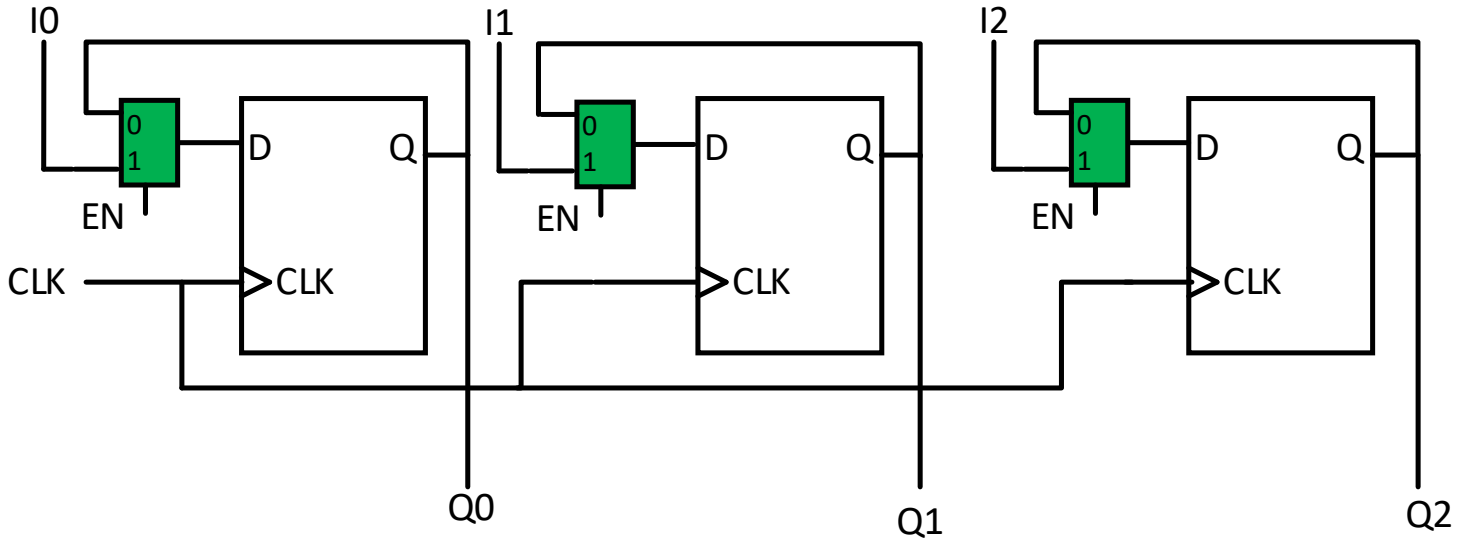
Registo

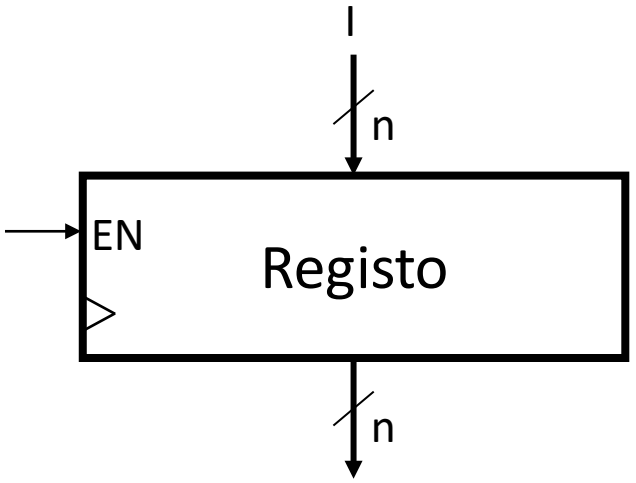
- Um registo de n-bits é um bloco de hardware capaz de memorizar n bits;
- Um registo n-bits é implementado com n flip-flops;
- Tal como os flip-flops, um registo pode ter entradas de set e de reset.



Registo com Enable

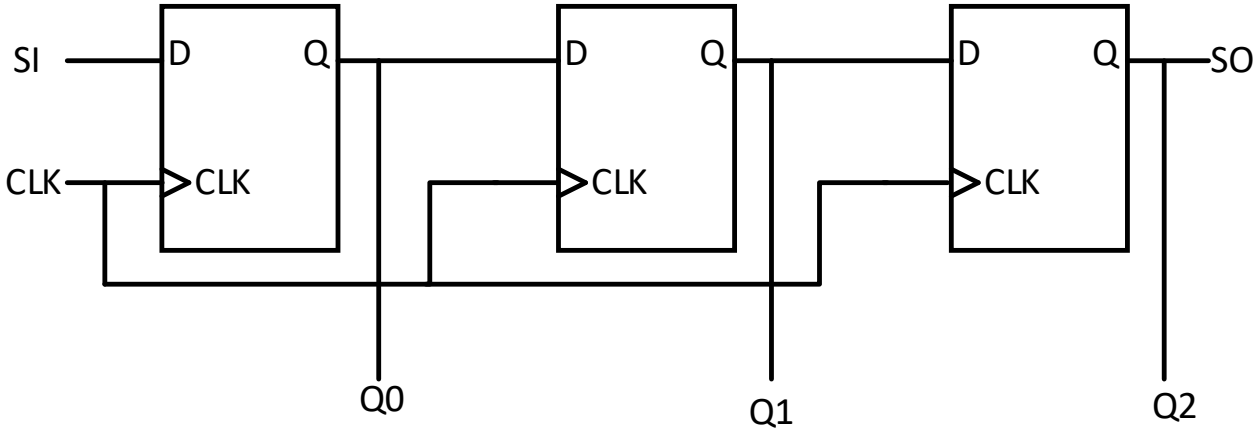
- Um registo com *enable* permite controlar se o registo mantém o valor ou atualiza o conteúdo com o valor à entrada;
- Realiza-se facilmente com um multiplexer à entrada de cada FF.



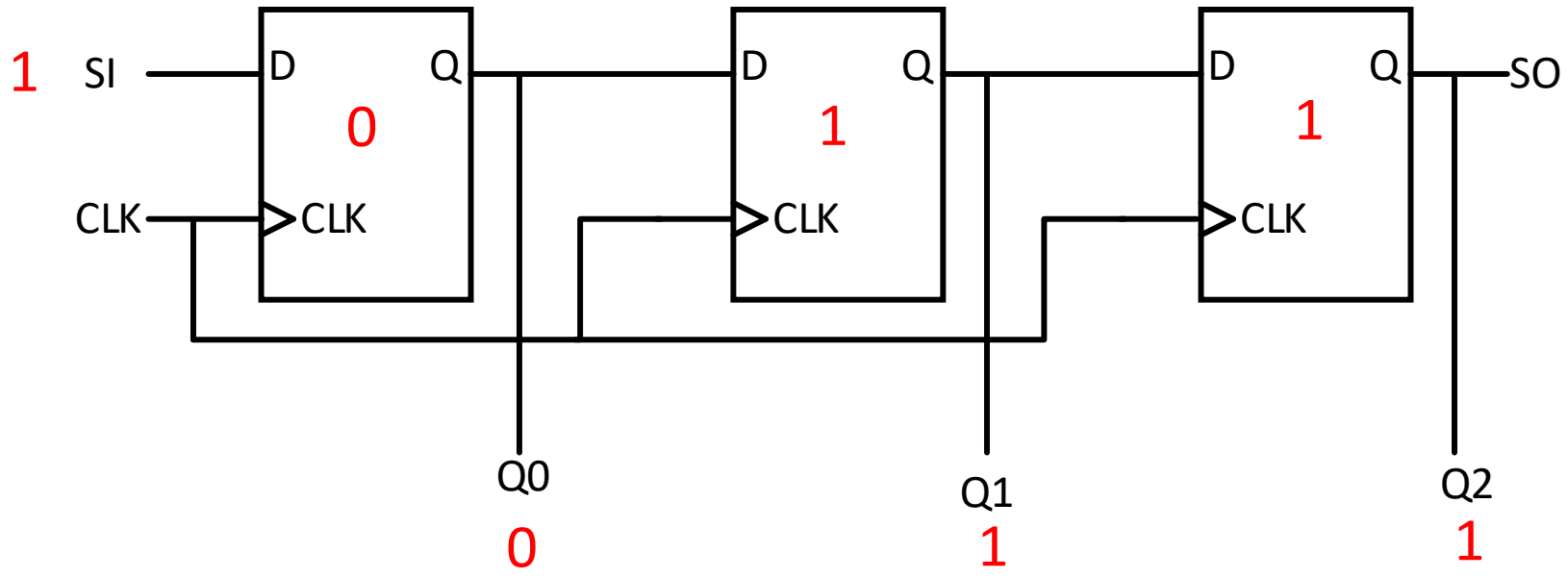


Registo de Deslocamento

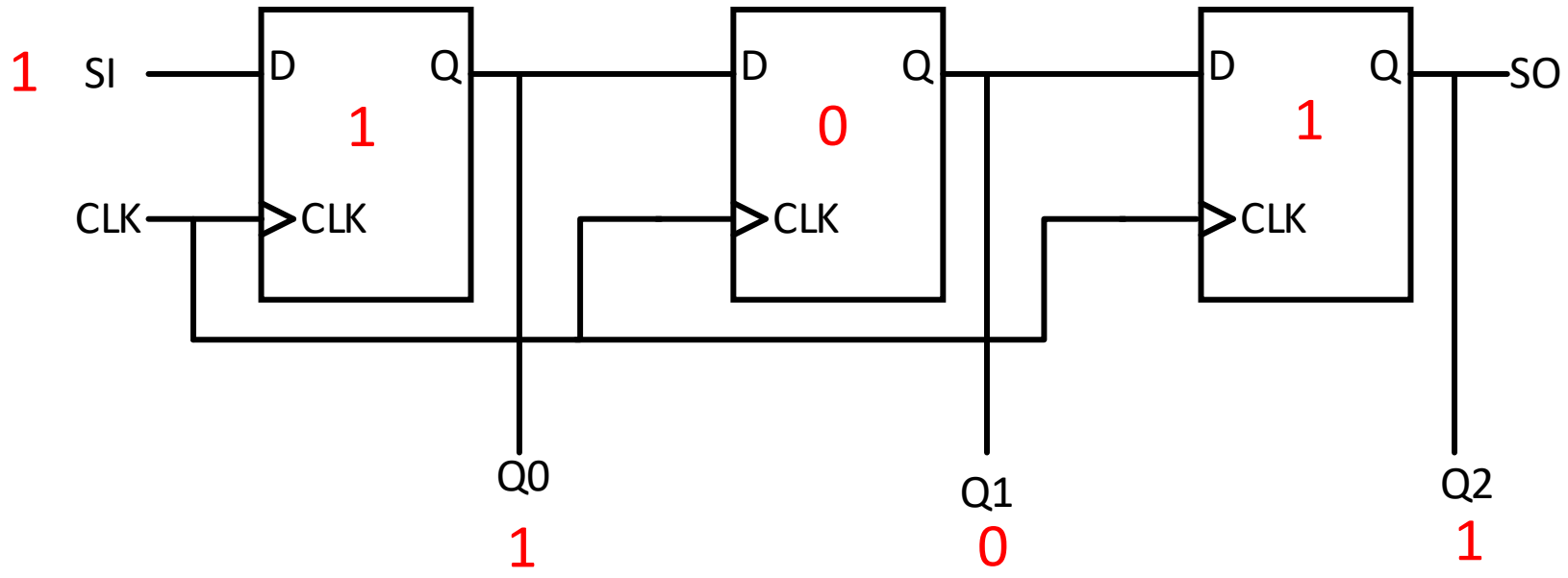
- Um registo de deslocamento permite a entrada em série dos dados (em vez de uma entrada em paralelo);
- Por cada ciclo de relógio, dá-se o deslocamento de um bit;
- Consideremos um registo de deslocamento à direita.



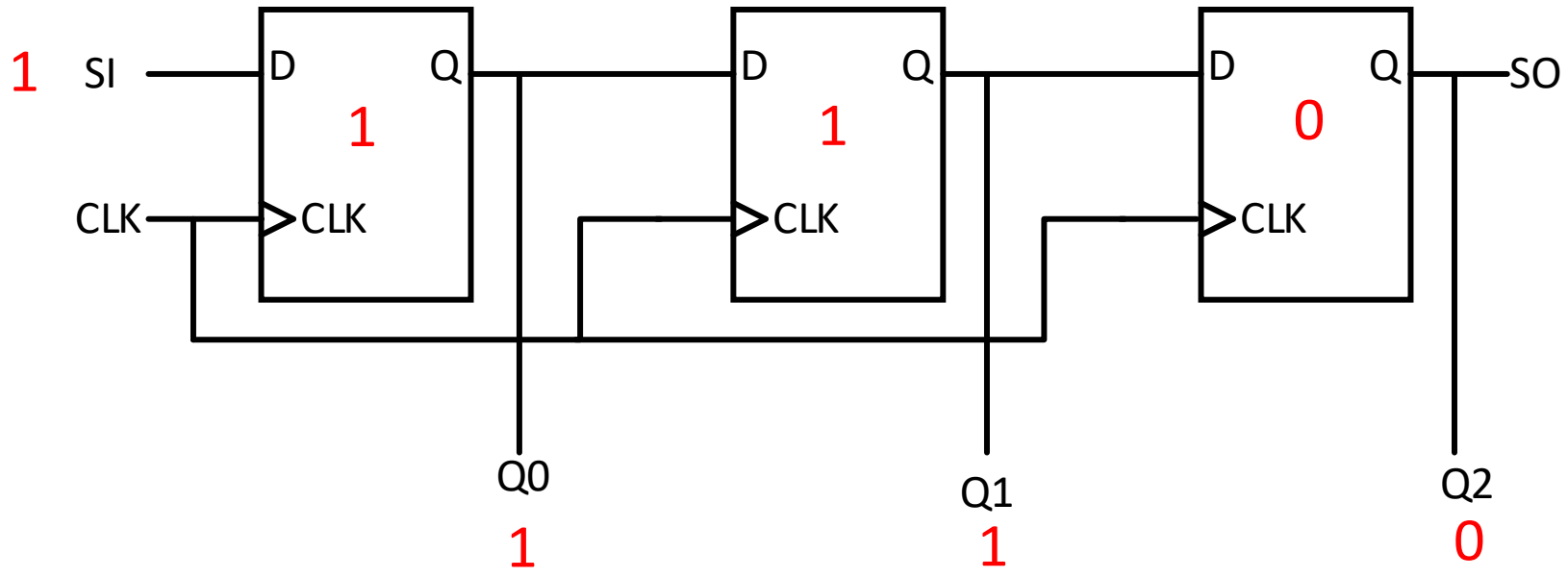
Registo de Deslocamento: Exemplo



Registo de Deslocamento: Exemplo

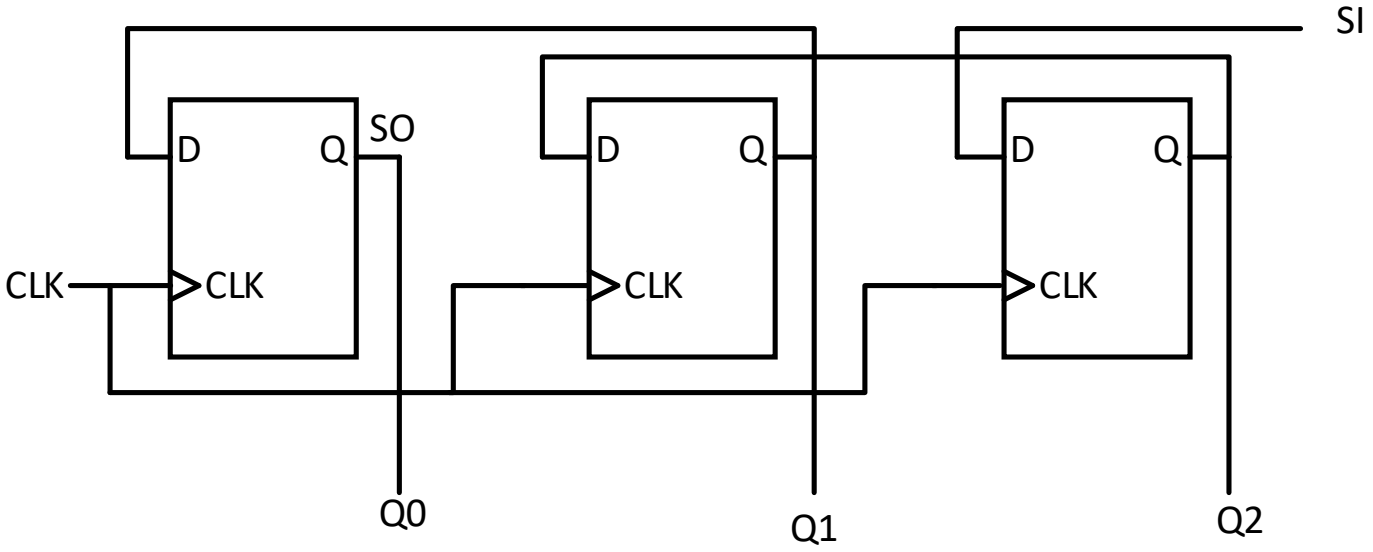


Registo de Deslocamento: Exemplo



Registo de Deslocamento à Esquerda

- Um registo de deslocamento pode ser realizado para a direita ou para esquerda;
- Consideremos um registo de deslocamento à esquerda.



Registo - VHDL

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY reg IS
PORT(	CLK : in std_logic;
		RESET : in STD_LOGIC;
		D : IN STD_LOGIC_VECTOR(3 downto 0);
		EN : IN STD_LOGIC;
		Q : out std_logic_VECTOR(3 downto 0)
		);
END reg;
```

Registo - VHDL

```
ARCHITECTURE logicFunction OF reg IS

component FFD
PORT(       CLK : in std_logic;
            RESET : in STD_LOGIC;
            SET : in std_logic;
            D : IN STD_LOGIC;
            EN : IN STD_LOGIC;
            Q : out std_logic);
end component;

BEGIN

U0: FFD port map(CLK => clk, reset => reset, set => '0', D => D(0), EN => EN, Q => Q(0));
U1: FFD port map(CLK => clk, reset => reset, set => '0', D => D(1), EN => EN, Q => Q(1));
U2: FFD port map(CLK => clk, reset => reset, set => '0', D => D(2), EN => EN, Q => Q(2));
U3: FFD port map(CLK => clk, reset => reset, set => '0', D => D(3), EN => EN, Q => Q(3));

END LogicFunction;
```

Contador

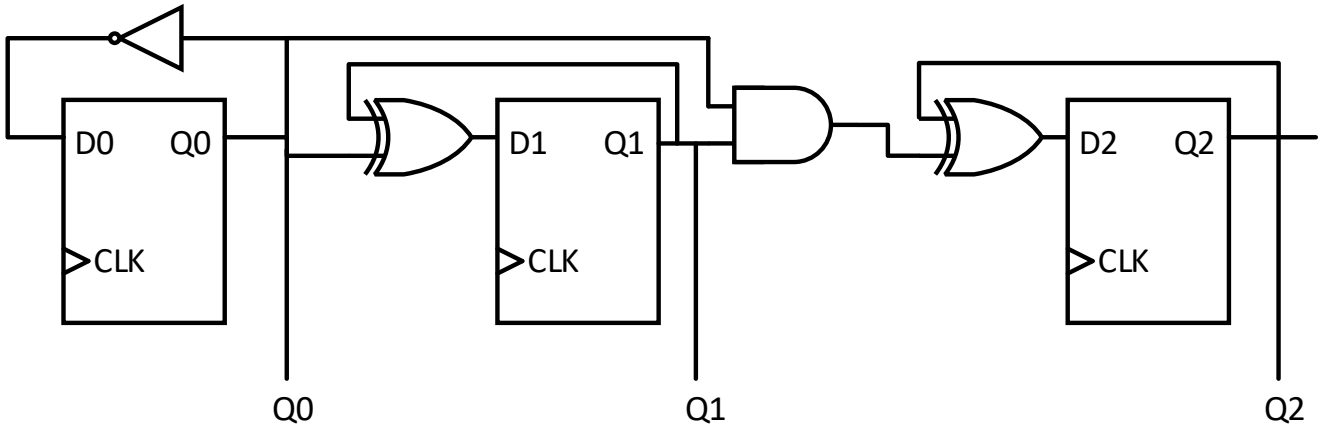
- Uma aplicação dos flip-flops é na implementação de contadores;
- Por cada impulso de relógio, a sequência de contagem evolui;
- Vamos apenas considerar contadores síncronos, em que todos os flip-flops são atuados pelo mesmo flanco de relógio;
- O módulo do contador é o número de contagem. Por exemplo, contador módulo 8 conta, por exemplo, de 0 a 7;

Contador Crescente com Flip-flops Tipo D

Um contador é uma máquina de estados, pelo que a sua síntese pode ser feita com os métodos de síntese de máquinas de estado estudados;

EP			ES		
Q2	Q1	Q0	D2	D1	D0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

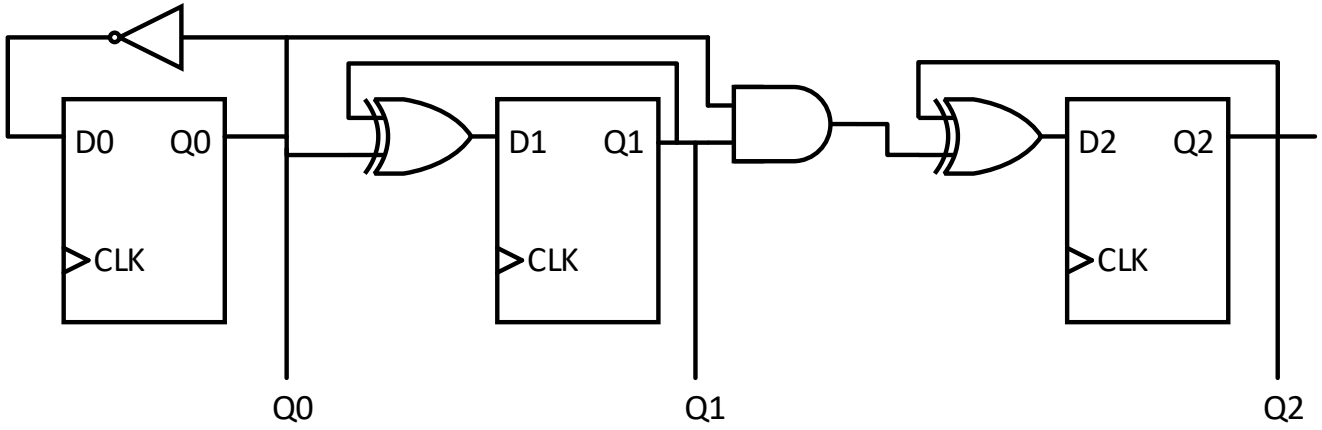
$$D0 = \overline{Q0}$$
$$D1 = Q1 \oplus Q0$$
$$D2 = (Q1\ Q0) \oplus Q2$$



Contador Crescente: Somador + Registo

Um contador é crescente é dado por X = X + 1;

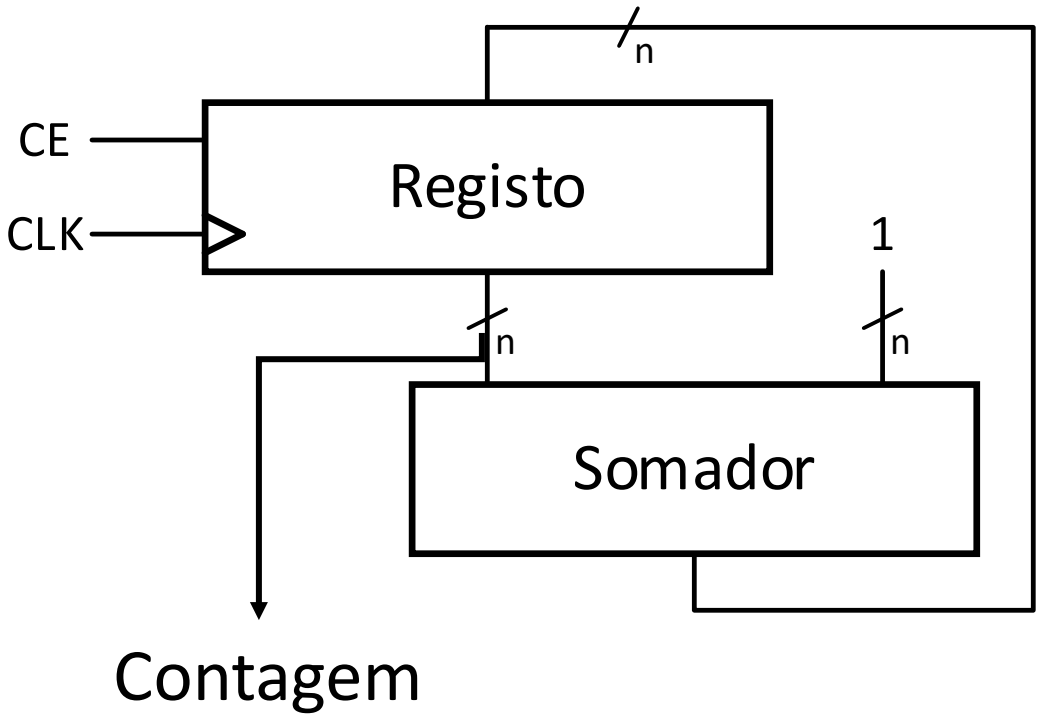
$$
\begin{array}{rccc}
 & Q1\ Q0 & Q0 & \\
 & Q2 & Q1 & Q0 \\
+ & 0 & 0 & 1 \\
\hline
 & Q1Q0 \oplus Q2 & Q1 \oplus Q0 & \overline{Q0}
\end{array}
$$



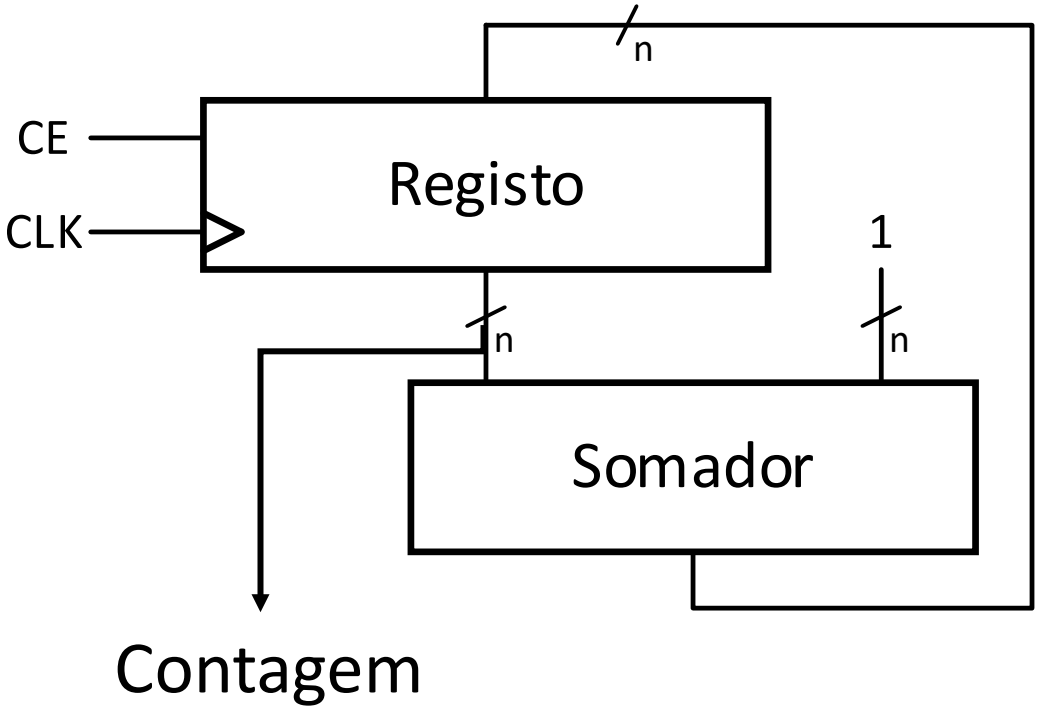
Contador Crescente: Somador + Registo

Um contador é crescente é dado por X = X + 1

Podemos parar a contagem com um *count enable* (CE)

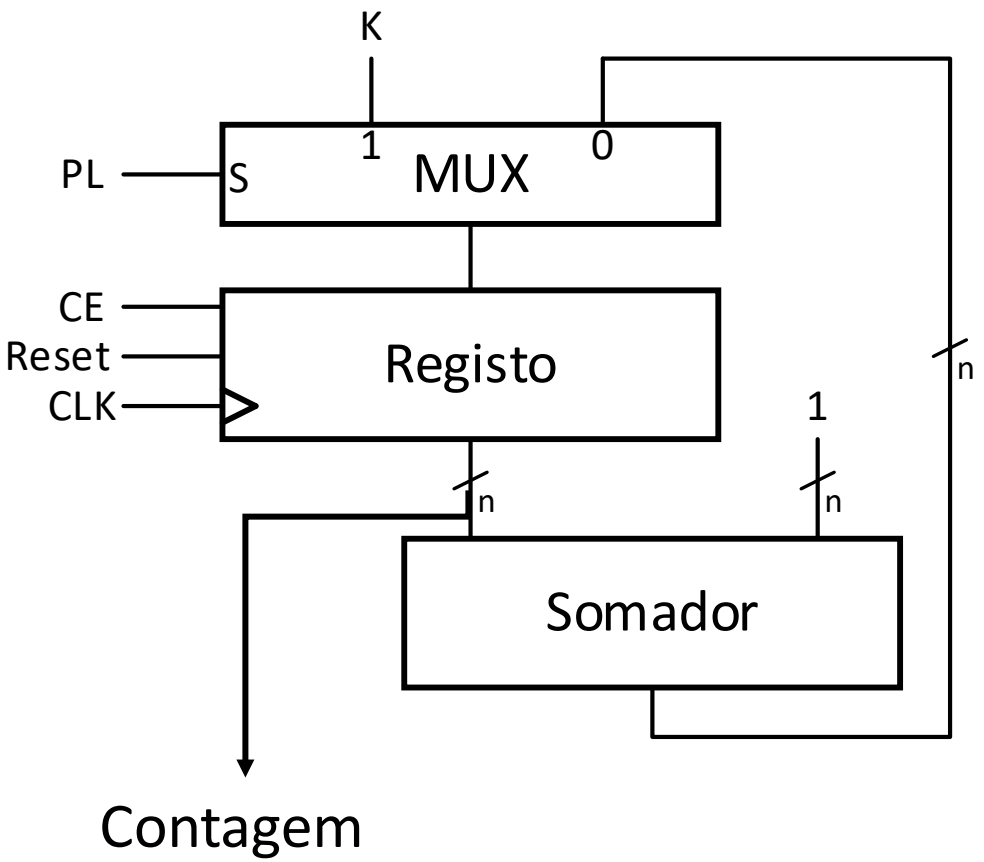


Como Iniciar o Contador?

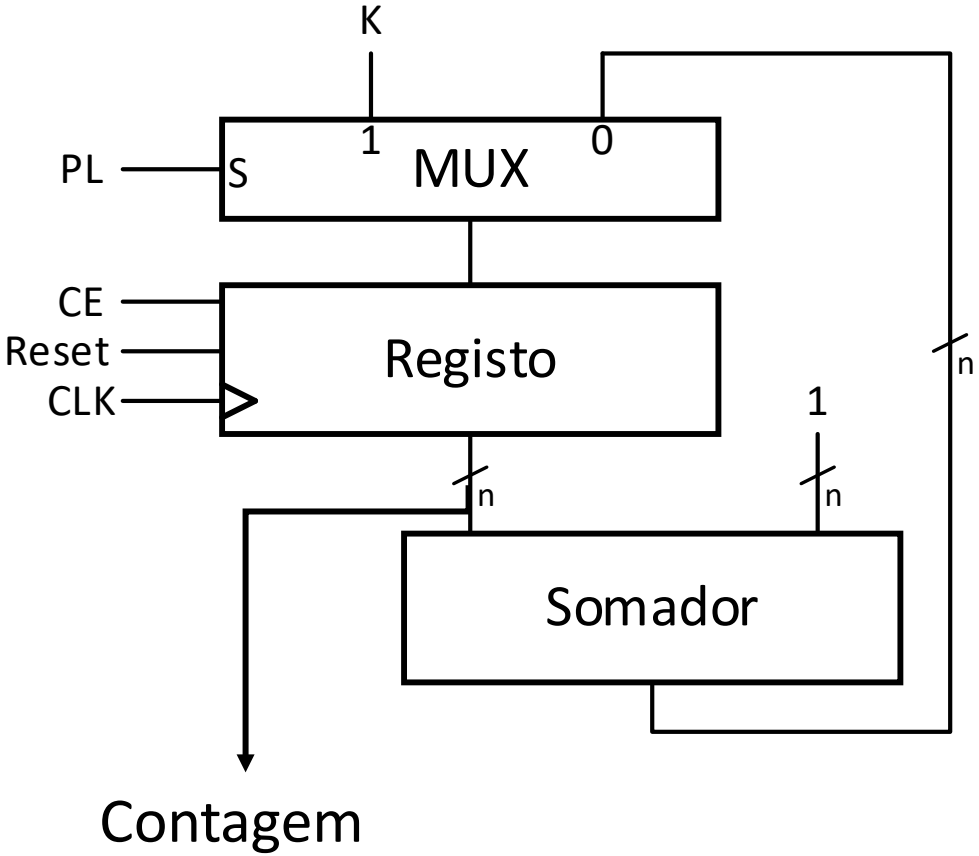


Contador com Carregamento Paralelo

- Iniciar o contador com uma determinada contagem:
  - Iniciar a 0 – Fazemos o Reset;
  - Iniciar a K – Selecionamos a entrada do registo com um multiplexer. Acrescenta-se uma entrada que indica se é feito carregamento paralelo ou se conta (PL – *Parallel Load*).



Como Contar entre K e M?

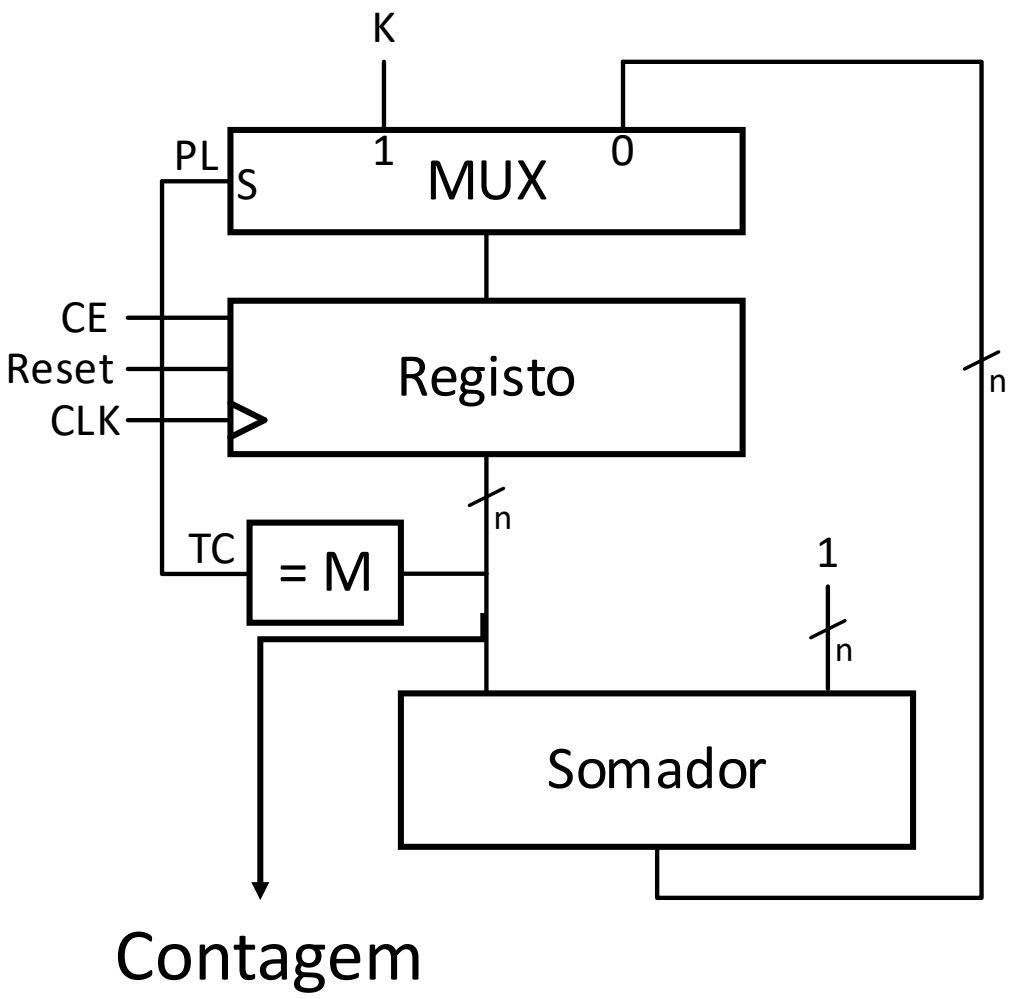


Contador com Módulo Arbitrário

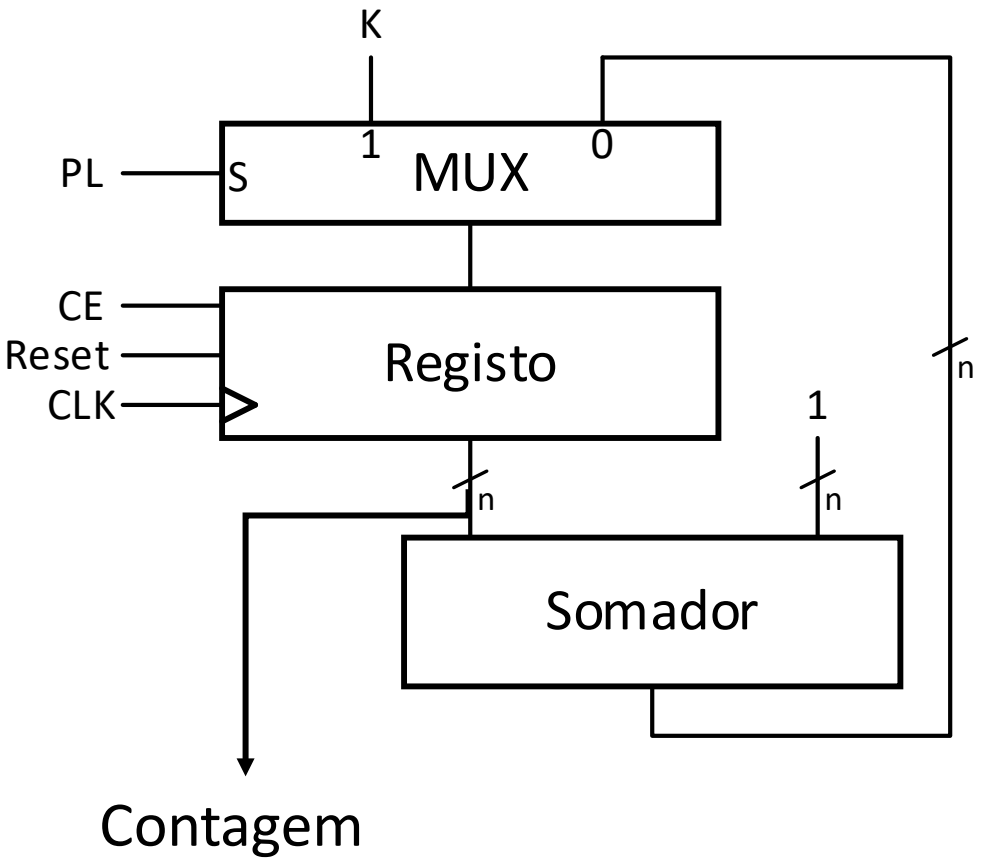
O contador pode ter um módulo arbitrário entre K e M;

Um comparador verifica o fim de contagem (*Terminal Count* - TC)

Quando contagem = M, faz carregamento paralelo de K

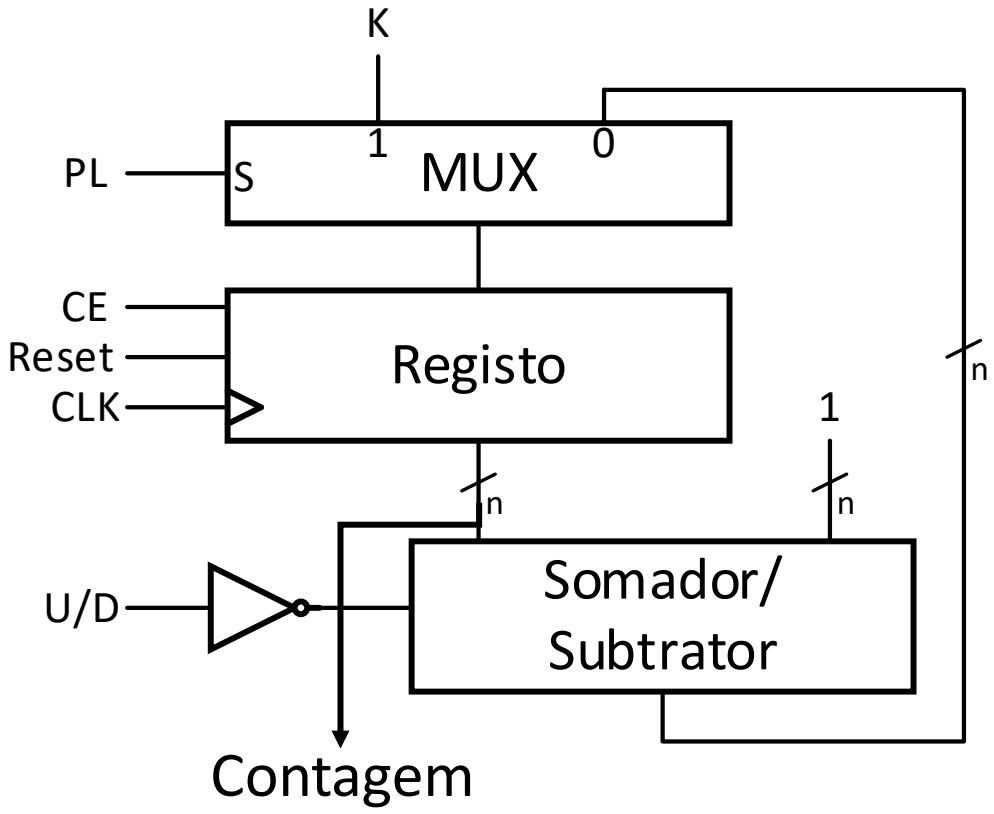


Contador UP/Down

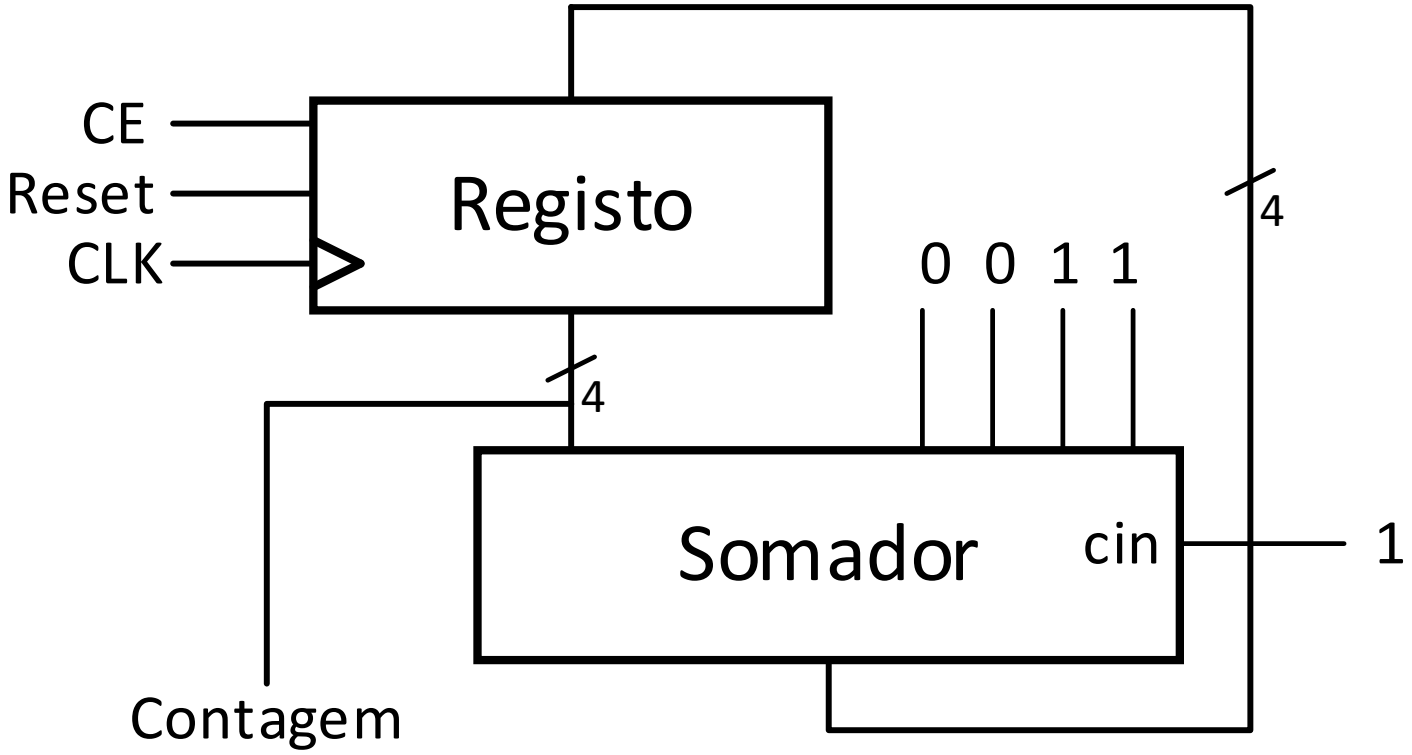


Contador UP/Down

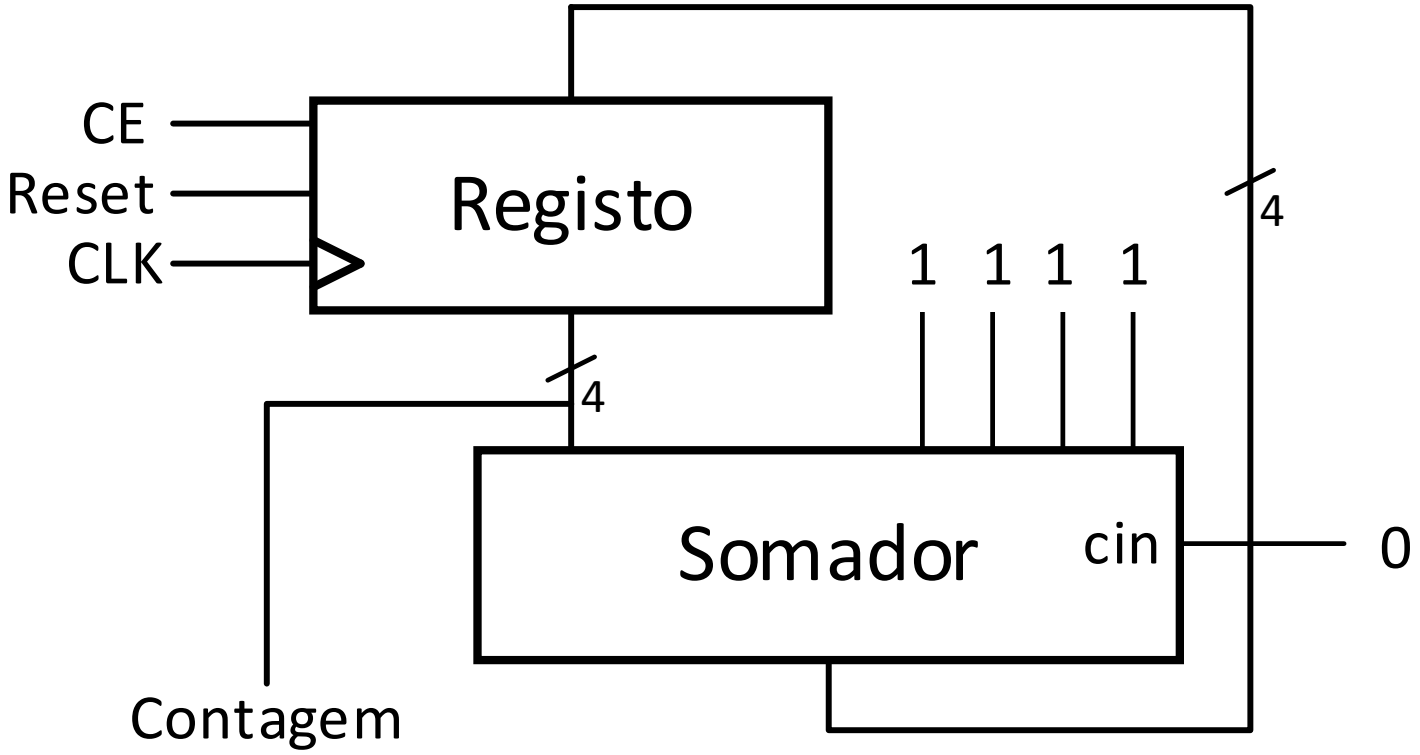
Uma variável indica se conta crescente (U_D = 1) ou decrescente (U_D = 0).



Qual a Sequência de Contagem?

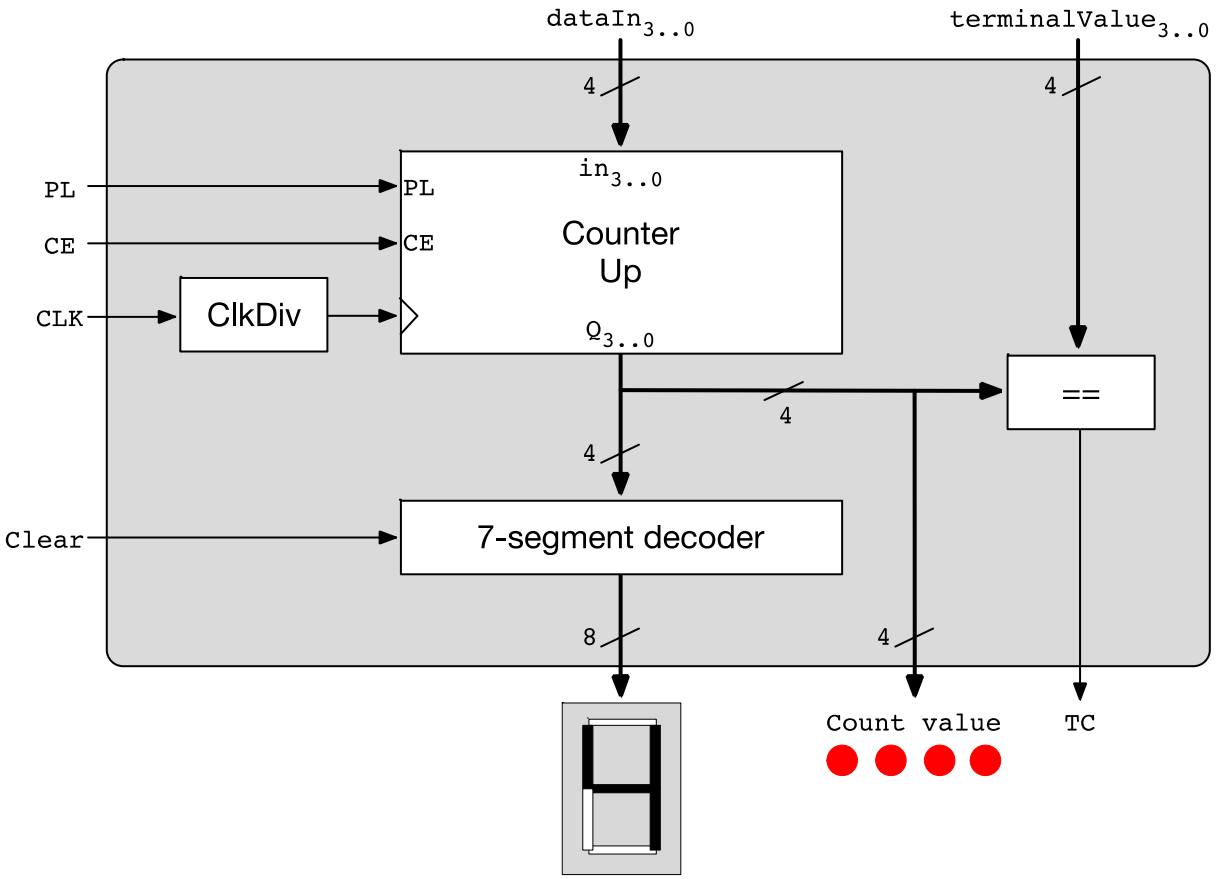


Qual a Sequência de Contagem?

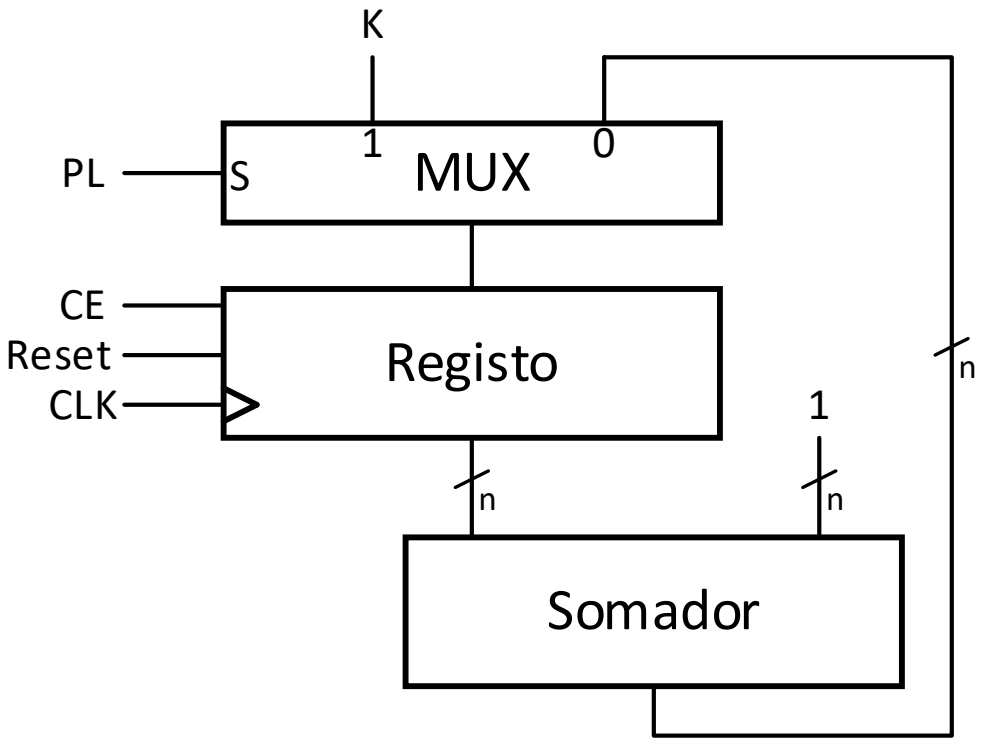


LABd

LABd - Contador



CounterUp



Registo - VHDL

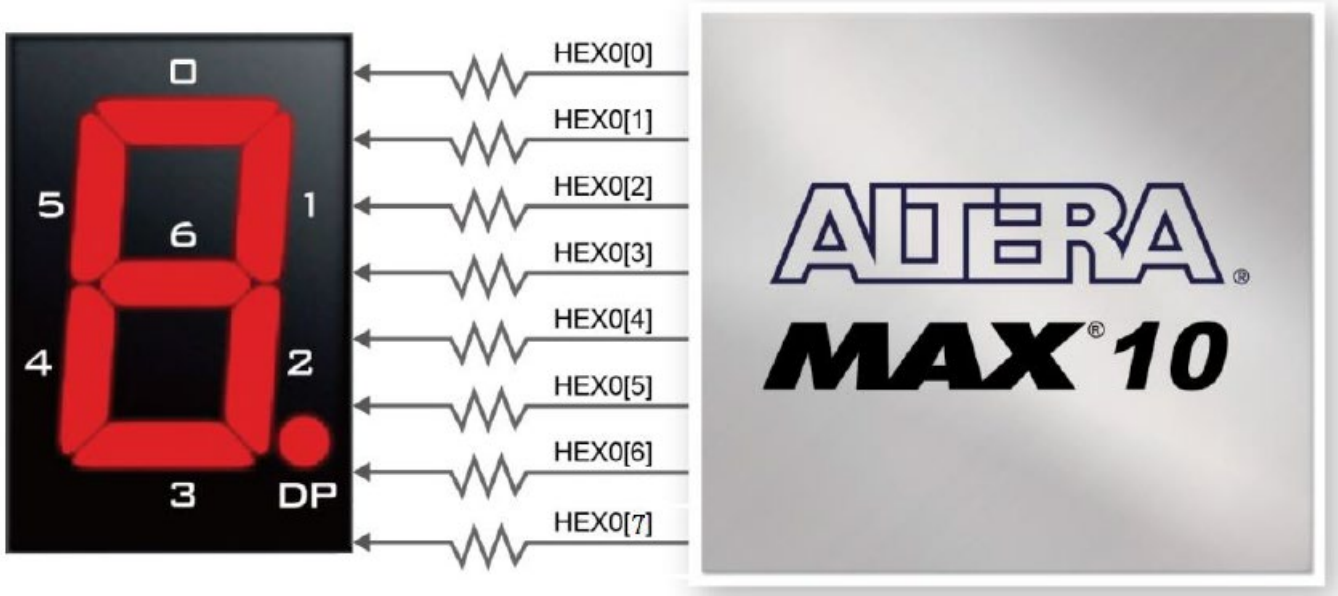
```
ENTITY reg IS
PORT(     CLK : in std_logic;
          RESET : in STD_LOGIC;
          D : IN STD_LOGIC_VECTOR(3 downto 0);
          EN : IN STD_LOGIC;
          Q : out std_logic_VECTOR(3 downto 0));
END reg;
```

Registo – VHDL (cont.)

```
ARCHITECTURE logicFunction OF reg IS

component FFD
PORT(     CLK : in std_logic;
          RESET : in STD_LOGIC;
          SET : in std_logic;
          D : IN STD_LOGIC;
          EN : IN STD_LOGIC;
          Q : out std_logic);
end component;
BEGIN

U0: FFD port map(CLK => clk, reset => reset, set => '0', D => D(0), EN => EN, Q => Q(0));
U1: FFD port map(CLK => clk, reset => reset, set => '0', D => D(1), EN => EN, Q => Q(1));
U2: FFD port map(CLK => clk, reset => reset, set => '0', D => D(2), EN => EN, Q => Q(2));
U3: FFD port map(CLK => clk, reset => reset, set => '0', D => D(3), EN => EN, Q => Q(3));

END LogicFunction;
```

7-Segment Decoder